



Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Innovative Applications of O.R.

Instance-dependent cost-sensitive learning for detecting transfer fraud

Sebastiaan Höppner^a, Bart Baesens^{b,c}, Wouter Verbeke^b, Tim Verdonck^{d,a,*}^a KU Leuven, Department of Mathematics, Celestijnenlaan 200B, Leuven 3001, Belgium^b KU Leuven, Faculty of Economics and Business, Naamsestraat 69, Leuven 3000, Belgium^c University of Southampton, School of Management, Highfield Southampton SO17 1BJ, United Kingdom^d University of Antwerp, Department of Mathematics, Middelheimlaan 1, Antwerp 2020, Belgium

ARTICLE INFO

Article history:

Received 1 May 2020

Accepted 18 May 2021

Available online xxx

Keywords:

Decision analysis

Fraud detection

Cost-based model evaluation

Cost-sensitive classification

ABSTRACT

Card transaction fraud is a growing problem affecting card holders worldwide. Financial institutions increasingly rely upon data-driven methods for developing fraud detection systems, which are able to automatically detect and block fraudulent transactions. From a machine learning perspective, the task of detecting fraudulent transactions is a binary classification problem. Classification models are commonly trained and evaluated in terms of statistical performance measures, such as likelihood and AUC, respectively. These measures, however, do not take into account the actual business objective, which is to minimize the financial losses due to fraud. Fraud detection is to be acknowledged as an instance-dependent cost-sensitive classification problem, where the costs due to misclassification vary between instances, and requiring adapted approaches for learning a classification model. In this article, an instance-dependent threshold is derived, based on the instance-dependent cost matrix for transfer fraud detection, that allows for making the optimal cost-based decision for each transaction. Two novel classifiers are presented, based on lasso-regularized logistic regression and gradient tree boosting, which directly minimize the proposed instance-dependent cost measure when learning a classification model. The proposed methods are implemented in the R packages *cslogit* and *csboost*, and compared against state-of-the-art methods on a publicly available data set from the machine learning competition website Kaggle and a proprietary card transaction data set. The results of the experiments highlight the potential of reducing fraud losses by adopting the proposed methods.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

In September 2018 the European Central Bank issued the fifth oversight report on card fraud. The report analyses developments in fraud that are related to card payment schemes (CPSs) in the Single Euro Payments Area (SEPA). The report indicates that the total value of fraudulent transactions conducted using cards issued within SEPA and acquired worldwide amounted to € 1.8 billion in 2016. In relative terms, i.e. as a share of the total value of transactions, the total value of fraudulent transfers amounted to 0.041% in 2016 (European Central Bank, September 2018). Therefore, developing powerful fraud detection systems is of crucial importance to financial institutions in order to reduce losses by timely blocking, containing and preventing fraudulent transactions.

A stream of literature has reported upon the adoption of data-driven approaches for developing fraud detection systems (Ngai, Hu, Wong, Chen, & Sun, 2011; Phua, Lee, Smith, & Gayler, 2010). Although these methods significantly improve the efficiency of fraud detection systems, opportunities exist to better align the development of data-driven fraud detection systems with the actual business objective. The objective that is adopted in learning from data by many data-driven approaches is *statistical* in nature, e.g., the likelihood or cross-entropy is maximized, whereas a more appropriate objective in learning would be to minimize the losses due to fraud.

For this purpose, cost-sensitive learning methods (Sahin, Bulkan, & Duman, 2013) may be adopted, which can take into account class-dependent misclassification costs (Chan & Stolfo, 1998). These methods are often adopted to address the class imbalance problem in fraud detection (Dal Pozzolo, Caelen, Le Borgne, Waterschoot, & Bontempi, 2014), since they allow to emphasize the importance of correctly identifying observations of the minority class, i.e. fraudulent transactions.

Recently, a number of methods that take into account transaction- or instance-dependent misclassification costs have

* Corresponding author at: KU Leuven, Department of Mathematics, Celestijnenlaan 200B, Leuven 3001, Belgium.

E-mail addresses: sebastiaan.hoppner@kuleuven.be (S. Höppner), bart.baesens@kuleuven.be (B. Baesens), Wouter.Verbeke@vub.be (W. Verbeke), Tim.Verdonck@uantwerpen.be (T. Verdonck).

Table 1

Confusion matrix of a binary classification task. Between square brackets, the related instance-dependent classification costs for transfer fraud are given.

	Actual legitimate (negative) $y = 0$	Actual fraudulent (positive) $y = 1$
Predicted as legitimate (negative) $\hat{y} = 0$	True negative [$C_i(0 0) = 0$]	False negative [$C_i(0 1) = A_i$]
Predicted as fraudulent (positive) $\hat{y} = 1$	False positive [$C_i(1 0) = c_f$]	True positive [$C_i(1 1) = c_f$]

been proposed and evaluated for detecting credit card fraud (Bahnsen, Aouada, & Ottersten, 2014a; Bahnsen et al., 2017). Alternative methods that aim at optimizing for the business objective while learning have been proposed that adopt a profit-driven strategy. These methods maximize the performance of the resulting fraud detection model as evaluated using a customized profit measure (Höppner, Stripling, Baesens, vanden Broucke, & Verdonck, 2018; Verbeke, Dejaeger, Martens, Hur, & Baesens, 2012).

In this paper, we further extend upon this recent stream of literature, by developing a theoretical underpinning for profit-driven, example-dependent learning, as well as by proposing and adopting a customized profit measure and objective functions for application in developing a data-driven credit card fraud detection system. More specifically, we introduce two novel instance-dependent cost-sensitive methods: cslogit and csboost, which are adapted from logistic regression and gradient tree boosting. Both methods adopt an objective function which assesses both the profitability of a model, by means of customized profit measure, and the complexity of a model, by means of a lasso penalty. Both methods have been implemented as R packages, including plot, summary and predict functions and will be published so as to allow reproduction of the results presented in this paper (with the exception of the results on a proprietary dataset). Both methods can be applied to any classification problem involving an instance- or class-dependent cost-matrix.

This paper is organized as follows. In Section 2, an instance-dependent cost-sensitive framework is introduced for making cost-optimal decisions with respect to card fraud detection. Section 2 also provides a theoretical underpinning for developing a customized objective function for data-driven learning, as implemented within logistic regression and gradient tree boosting in Section 3. Section 4 presents the results of an empirical evaluation of the proposed approaches. Finally, concluding remarks and potential directions for future research are provided in Section 5.

2. Instance-dependent cost-sensitive framework for transfer fraud detection

2.1. Instance-dependent cost matrix

The aim of detecting transfer fraud is to identify transactions with a high probability of being fraudulent. From the perspective of machine learning, the task of predicting the fraudulent nature of transactions can be presented as a binary classification problem where instances belong either to class 0 or to class 1. We follow the convention that the instances of interest such as fraudulent transactions, belong to class 1, whereas the other instances such as legitimate transfers, correspond to class 0. We often speak of positive (class 1) and negative (class 0) instances.

In general, a classification exercise leads to a confusion matrix as shown in Table 1. For example, the upper right cell contains the instances belonging to class 1 (e.g. fraudulent transactions) which are incorrectly classified into class 0 (e.g. predicted as being legitimate).

The outcome of a classification task is usually related to costs for incorrect classifications and benefits for correctly classified instances. Let $C_i(\hat{y}|y)$ be the cost of predicting class $\hat{y}$ for an instance i when the true class is y (i.e. $y, \hat{y} \in \{0, 1\}$). If $\hat{y} = y$ then the prediction is correct, while if $\hat{y} \neq y$ the prediction is incorrect. In general, the costs can be different for each of the four cells in the confusion matrix and can even be instance-dependent, in other words, specific to each transaction i as indicated in Table 1. Hand, Whitrow, Adams, Juszczak, and Weston (2008) proposed a cost matrix, where in the case of a false positive (i.e. incorrectly predicting a transaction as fraudulent) the associated cost is the administrative cost $C_i(1|0) = c_f$. This fixed cost c_f has to do with investigating the transaction and contacting the card holder. When detecting a fraudulent transfer, the same cost $C_i(1|1)$ is allocated to a true positive, because in this situation, the card owner will still need to be contacted. In other words, the action undertaken by the company towards an individual transaction i comes at a fixed cost $c_f \geq 0$, regardless of the nature of the transaction. However, in the case of a false negative, in which a fraudulent transfer is not detected, the cost is defined to be the amount $C_i(0|1) = A_i$ of the transaction i . The instance-dependent costs are summarized in Table 1. We argue that the proposed cost matrix in Table 1 is a reasonable assumption. However, the framework that is presented in this paper, including the algorithms cslogit and csboost, can deal with any cost matrix. For example, rather than using a fixed cost for false positives, one could choose to incorporate a variable cost that reflects the level of friction that the card holder experiences.

2.2. Making optimal cost-based decisions

Given the cost specification for correct and incorrect predictions, an instance should be predicted to have the class leading to the smallest expected loss (Elkan, 2001). Here the expectation is calculated using the conditional probability of each class given the instance. These conditional probabilities are estimated by a classification algorithm. In general, a classification algorithm models the relation between d explanatory variables $\mathbf{X} = (X_1, \dots, X_d)$ and the binary response variable $Y \in \{0, 1\}$. Such a model can be used to predict the fraud propensity of transactions on the basis of their observed variables $\mathbf{x} \in \mathbf{X}$. In particular, a classification algorithm is a function that models the conditional expected value of Y :

$$s : \mathbf{X} \rightarrow [0, 1] : \mathbf{x} \mapsto s(\mathbf{x}) = E(Y|\mathbf{x}) = P(Y = 1|\mathbf{x}).$$

Thus, a classification algorithm provides a continuous score $s_i := s(\mathbf{x}_i) \in [0, 1]$ for each transaction i . This score s_i is a function of the observed features $\mathbf{x}_i$ of transaction i and presents the fraud propensity of that transaction. Here we assume that legitimate transfers (class 0) have a lower score than fraudulent ones (class 1).

The optimal cost-based prediction for transaction i is the class $\hat{y}$ that minimizes its expected loss,

$$\begin{aligned} EL(\mathbf{x}_i, \hat{y}) &= \sum_y P(Y = y|\mathbf{x}_i)C_i(\hat{y}|y) \\ &= P(Y = 0|\mathbf{x}_i)C_i(\hat{y}|0) + P(Y = 1|\mathbf{x}_i)C_i(\hat{y}|1). \end{aligned} \quad (1)$$

The role of a classification algorithm is to estimate the probability $P(Y = y|\mathbf{x}_i)$ for each transaction i where y is the true class of the transaction. The optimal prediction for a transaction is class 1 (fraud) if and only if the expected loss of this prediction is less than the expected loss of predicting class 0 (legitimate), i.e. if and only if

$$\begin{aligned} &EL(\mathbf{x}_i, \hat{y} = 1) < EL(\mathbf{x}_i, \hat{y} = 0) \\ \Leftrightarrow &P(Y = 0|\mathbf{x}_i)C_i(1|0) + P(Y = 1|\mathbf{x}_i)C_i(1|1) \\ &< P(Y = 0|\mathbf{x}_i)C_i(0|0) + P(Y = 1|\mathbf{x}_i)C_i(0|1) \\ \Leftrightarrow &(1 - s_i)C_i(1|0) + s_iC_i(1|1) < (1 - s_i)C_i(0|0) + s_iC_i(0|1) \\ &C_i(1|0) - C_i(0|0) \\ \Leftrightarrow &s_i > \frac{C_i(1|0) - C_i(0|0)}{C_i(1|0) - C_i(0|0) + C_i(0|1) - C_i(1|1)} \end{aligned}$$

given $s_i = P(Y = 1|\mathbf{x}_i)$. Thus, using the costs for transfer fraud as indicated in Table 1, the threshold for making the optimal decision for a transaction i is

$$t_i^* = \frac{C_i(1|0) - C_i(0|0)}{C_i(1|0) - C_i(0|0) + C_i(0|1) - C_i(1|1)} = \frac{c_f}{A_i} \quad (2)$$

assuming that the transferred amount A_i is nonzero. In conclusion, the optimal prediction for transaction i with score s_i is class 1 (fraud) if and only if $s_i > t_i^* = c_f/A_i$, while transfer i is predicted as legitimate if and only if $s_i \leq t_i^* = c_f/A_i$. Making the prediction $\hat{y}$ for a transaction implies acting as if $\hat{y}$ is the true class of that transaction. Note that the s_i , as estimated by the classification model, are assumed to be calibrated probabilities rather than just scores that rank the transfers from most suspicious to least (Bahnsen, Stojanovic, Aouada, & Ottersten, 2014b; Zadrozny & Elkan, 2001). This is especially important when making a decision based on these probabilities and their respective threshold.

The essence of cost-sensitive decision making is that, even when some class is more probable, it can be more profitable to act as if another class is true. For example, it can be rational to block a large transaction even if the transaction is most likely legitimate. Consider, for example, a transaction of € 1000 with a small estimated fraud propensity of 10%. If the transaction is classified as legitimate, the expected loss is € 100 as, on average, 1 in 10 of these kind of transactions is in fact expected to be fraudulent. On the other hand, if the transaction is classified as fraudulent, the expected loss is the administrative cost c_f , for example € 10. Therefore, this transaction will be treated as fraudulent despite its fraud probability being only 10%. While it may be of interest to a financial institution to minimize false positives, the ultimate goal of the company is to maximize profits which is better addressed by the minimization of the financial costs. By using a fraud detection system, the financial institution will be able to identify fraudulent transfers and thus prevent money from being stolen from its customers, hereby reducing losses and thus generating a profit as compared to accepting or rejecting all transactions.

2.3. Cost of a fraud detection model

Let $\mathcal{D}$ denote a set of N transactions which consists of the observed predictor-response pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, where $y_i \in \{0, 1\}$ describes the binary response and $\mathbf{x}_i = (x_{i1}, \dots, x_{id})$ represents the d associated predictor variables of transaction i . A classification model $s(\cdot)$ is trained on the set $\mathcal{D}$ such that it generates a score or fraud propensity $s_i \in [0, 1]$ for each transaction i based on the observed features $\mathbf{x}_i$ of the transfer. The score s_i is then converted to a predicted class $\hat{y}_i \in \{0, 1\}$ by comparing it with its optimal classification threshold t_i^* (2). The cost of using $s(\cdot)$ on the transactions of $\mathcal{D}$ is calculated by (Bahnsen, Aouada, Stojanovic, & Ottersten, 2016)

$$\begin{aligned} \text{Cost}(s(\mathcal{D})) &= \sum_{i=1}^N \left(y_i \left[\hat{y}_i C_i(1|1) + (1 - \hat{y}_i) C_i(0|1) \right] \right. \\ &\quad \left. + (1 - y_i) \left[\hat{y}_i C_i(1|0) + (1 - \hat{y}_i) C_i(0|0) \right] \right) \quad (3) \\ &= \sum_{i=1}^N y_i (1 - \hat{y}_i) A_i + \hat{y}_i c_f. \end{aligned}$$

In other words, the total cost is the sum of the amounts of the undetected fraudulent transactions ($y_i = 1, \hat{y}_i = 0$) plus the administrative cost incurred. The total cost may not always be easy to interpret because there is no reference to which the cost is compared (Whitrow, Hand, Juszczak, Weston, & Adams, 2009). So Bahnsen et al. (2016) proposed the *cost savings* of a classification algorithm as the cost of using the algorithm compared to using no algorithm at all. The cost of using no algorithm is

$$\text{Cost}_l(\mathcal{D}) = \min\{\text{Cost}(s_0(\mathcal{D})), \text{Cost}(s_1(\mathcal{D}))\} \quad (4)$$

where s_0 refers to a classifier that predicts all the transactions in $\mathcal{D}$ as belonging to class 0 (legitimate) and similarly s_1 refers to a classifier that predicts all the transfers in $\mathcal{D}$ as belonging to class 1 (fraud).

The cost savings is then expressed as the cost improvement of using an algorithm as compared with $\text{Cost}_l(\mathcal{D})$,

$$\text{Savings}(s(\mathcal{D})) = \frac{\text{Cost}_l(\mathcal{D}) - \text{Cost}(s(\mathcal{D}))}{\text{Cost}_l(\mathcal{D})}. \quad (5)$$

In the case of credit card transaction fraud, the cost of not using an algorithm is equal to the sum of amounts of the fraudulent transactions, $\text{Cost}_l(\mathcal{D}) = \sum_{i=1}^N y_i A_i$. The savings are then calculated as

$$\text{Savings}(s(\mathcal{D})) = \frac{\sum_{i=1}^N y_i \hat{y}_i A_i - \hat{y}_i c_f}{\sum_{i=1}^N y_i A_i}. \quad (6)$$

In other words, the costs that can be saved by using an algorithm are the sum of amounts of detected fraudulent transactions minus the administrative cost incurred in detecting them, divided by the sum of amounts of the fraudulent transactions. If $c_f = 0$, then the cost savings is the proportion of amounts of fraudulent transactions that are detected.

Notice that $\text{Cost}(s(\mathcal{D}))$ in (3) depends on the optimal threshold t_i^* for each transaction i through the prediction $\hat{y}_i$. The conditional expected value of $\hat{y}_i$ is given by

$$E[\hat{y}_i | \mathbf{x}_i] = P(\hat{y}_i = 1 | \mathbf{x}_i) \approx s(\mathbf{x}_i).$$

Therefore, we define the *average expected cost* (AEC) of a classification model $s(\cdot)$ on a set $\mathcal{D}$ as

$$\begin{aligned} \text{AEC}(s(\mathcal{D})) &= \frac{1}{N} E[\text{Cost}(s(\mathcal{D})) | \mathbf{X}] \\ &= \frac{1}{N} \sum_{i=1}^N \left(y_i \left[s_i C_i(1|1) + (1 - s_i) C_i(0|1) \right] \right. \\ &\quad \left. + (1 - y_i) \left[s_i C_i(1|0) + (1 - s_i) C_i(0|0) \right] \right) \\ &= \frac{1}{N} \sum_{i=1}^N y_i (1 - s_i) A_i + s_i c_f. \quad (7) \end{aligned}$$

Notice that the average expected cost (AEC) is independent of any threshold value because it relies on the estimated probabilities s_i instead of the predicted classes $\hat{y}_i$. If the transfer is fraudulent ($y_i = 1$), then $y_i(1 - s_i)A_i$ is the estimated fraction of the fraudulent amount that is not detected and thus it is the expected cost of that transaction. The term $s_i c_f$ is the expected fraction of the fixed cost c_f that is incurred for transfer i . Similarly, the *expected savings* are computed as

$$\begin{aligned} \text{Expected Savings}(s(\mathcal{D})) &= E[\text{Savings}(s(\mathcal{D})) | \mathbf{X}] \\ &= \frac{\text{Cost}_l(\mathcal{D}) - E[\text{Cost}(s(\mathcal{D})) | \mathbf{X}]}{\text{Cost}_l(\mathcal{D})} \\ &= \frac{\sum_{i=1}^N y_i s_i A_i - s_i c_f}{\sum_{i=1}^N y_i A_i}. \quad (8) \end{aligned}$$

Remarks: In the derivation of Eqs. (6) and (8), we assume that $\text{Cost}_l(\mathcal{D}) = \text{Cost}(s_0(\mathcal{D}))$. This holds if and only if

$$\text{Cost}(s_0(\mathcal{D})) < \text{Cost}(s_1(\mathcal{D})) \Leftrightarrow \sum_{i=1}^N y_i A_i < \sum_{i=1}^N c_f \Leftrightarrow c_f > \frac{1}{N} \sum_{i=1}^N y_i A_i. \quad (9)$$

Therefore, in order for Eqs. (6) and (8) to hold, the lower bound on the fixed cost c_f with respect to a set of transactions $\mathcal{D}$ is equal

to the total of fraudulent amounts in $\mathcal{D}$ divided by the number of transactions N in set $\mathcal{D}$. If c_f is too small, then the cost of blocking a transaction is almost negligible and thus it is less costly to block and investigate each transaction for fraud rather than letting each transaction pass and incurring financial costs due to fraud. This almost never holds since most banks may process over 100,000 transactions each day. Therefore, one should not choose c_f too small.

Given a certain value for the fixed cost c_f , the optimal decision threshold for a transaction $t_i^* = c_f/A_i$ will be larger or equal to 1 if $A_i \leq c_f$. Therefore, a transaction will never be blocked if the transferred amount is below the fixed cost since then its score s_i will always be below its threshold $t_i^* = 1$. It is in the interest of the bank or transfer-processing company to keep the value of the fixed cost confidential since fraudsters may abuse this information to attempt to make many fraudulent transactions with an amount slightly below the fixed cost value without getting caught.

Concerning the choice of the value for the fixed cost, there are two possible approaches. The fixed cost can be determined by business experts who calculate the cost of investigating a transaction and contacting the card holder. However, in practice it is not straightforward to find out this true administrative cost. The fixed cost c_f can also be regarded as a parameter for tuning the decision thresholds t_i^* . For example, the business may require that at least 80% of fraudulent transfers are blocked and the fixed cost thus can be tuned accordingly.

3. Cost-sensitive logistic regression and gradient tree boosting

Popular methods for dealing with binary classification problems include logistic regression and gradient tree boosting. We opt to use logistic regression because it is widely used in the industry, it is fast to compute, easy to understand and interpret, and its flexible model structure allows for straightforward modification. Moreover, logistic regression is often used as a benchmark model to which other classification algorithms are compared. A single decision tree model is interpretable as well, but performance is observed to be less stable across datasets. Decision tree ensemble methods, such as boosting, typically yield strong predictive power but result in a black box model. Moreover, tree boosting is a machine learning technique which is used widely by data scientists to achieve state-of-the-art results on many machine learning challenges and has been used by a series of competition winning solutions (Chen & Guestrin, 2016). Therefore, we also adapt an algorithm for gradient tree boosting to the framework of instance-dependent costs.

3.1. Logistic regression

In general, a classification algorithm models the conditional mean of the binary response variable Y :

$$s_{\theta} : \mathbf{X} \rightarrow [0, 1] : \mathbf{x} \mapsto s_{\theta}(\mathbf{x}) = E(Y|\mathbf{x}) = P(Y = 1|\mathbf{x})$$

where $\theta \in \Theta$ are the model parameters and Θ is the parameter space. Logistic regression is a classification model that estimates the conditional probability $P(Y = 1|\mathbf{x})$ of the positive class (fraud) as the logistic sigmoid of a linear function of the feature vector $\mathbf{x}$ (Hosmer Jr, Lemeshow, & Sturdivant, 2013). The fraud propensity of a transaction is modeled as

$$s_{(\beta_0, \beta)}(\mathbf{x}) = P(Y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta^T \mathbf{x})}}. \quad (10)$$

Here, the model parameters are $\theta = (\beta_0, \beta)$ where $\beta_0 \in \mathbb{R}$ represents the intercept and $\beta \in \mathbb{R}^d$ is the d -dimensional vector of regression coefficients. The problem then becomes finding the optimal values for the parameters that optimize a given objective (i.e.

loss) function. The objective function is defined to measure the performance of a classification algorithm, given its parameters θ , on the data $(Y, \mathbf{X})$:

$$Q_{Y, \mathbf{X}} : \Theta \rightarrow \mathbb{R} : \theta \mapsto Q_{Y, \mathbf{X}}(\theta).$$

Usually, in the case of logistic regression, the optimal model parameters are the ones that minimize the binomial negative log-likelihood function,

$$Q_{Y, \mathbf{X}}^l(\beta_0, \beta) = -\frac{1}{N} \sum_{i=1}^N y_i \log(s_i) + (1 - y_i) \log(1 - s_i) \quad (11)$$

where we use $s_i = s_{(\beta_0, \beta)}(\mathbf{x}_i)$ to simplify the notation. Since this objection function is convex (Murphy, 2012), it is generally optimized using a gradient descent algorithm like the Newton-Raphson approach. However, this objective function assigns the same weight to both false positives and false negatives. As discussed before, this is not the case in many real-world applications, including credit card transaction fraud. Instead, the weighted log-likelihood function includes a weight w_i to each instance i in the likelihood function depending on the instance's class (positive or negative class) or on the instance itself:

$$Q_{Y, \mathbf{X}}^w(\beta_0, \beta) = -\frac{1}{N} \sum_{i=1}^N w_i [y_i \log(s_i) + (1 - y_i) \log(1 - s_i)].$$

For example, the weight assigned to an observation can be its relative cost, $w_i = c_i / \sum_{j=1}^n c_j$, where c_i is the cost of misclassifying observation i (e.g. $C_i(1|0)$ or $C_i(0|1)$). However, rather than optimizing a (weighted) likelihood function, the actual business objective is the minimize financial losses due to fraud and this should be reflected in the method's objective function. To that end, an instance-dependent cost-sensitive logistic model can be obtained by using the average expected cost (7) as objective function because it incorporates the different classification costs from Table 1,

$$Q_{Y, \mathbf{X}}^c(\beta_0, \beta) = AEC(\beta_0, \beta). \quad (12)$$

Notice that $Q_{Y, \mathbf{X}}^c$ depends on the regression coefficients (β_0, β) through the scores s_i . The optimal regression parameters are the values that minimize $AEC(\beta_0, \beta)$. To find these optimal regression coefficients, the AEC is minimized by using the gradient-based optimization method by Kraft (1988, 1994), called the sequential quadratic programming algorithm. The components of the gradient of $AEC(\beta_0, \beta)$ are given by the following partial derivatives:

$$\begin{aligned} \frac{\partial AEC(\beta_0, \beta)}{\partial \beta_j} &= \frac{1}{N} \sum_{i=1}^N x_{ij} s_i (1 - s_i) \left[y_i (C_i(1|1) - C_i(0|1)) \right. \\ &\quad \left. + (1 - y_i) (C_i(1|0) - C_i(0|0)) \right] \\ &= \frac{1}{N} \sum_{i=1}^N x_{ij} s_i (1 - s_i) (c_f - y_i A_i) \quad (j = 0, 1, \dots, d) \end{aligned} \quad (13)$$

where the design matrix $\mathbf{X}$ is defined such that its first column consists of ones, i.e. $x_{ij} = 1$ for $j = 0$. Notice that the gradient of the AEC can be easily computed due to the choice of using logistic regression (10) to model the fraud propensities s_i .

In an effort to identify potential mechanisms to improve the performance, we conducted analyses of $AEC(\beta_0, \beta)$ as an objective function for logistic regression like in Fig. 1a. We found that a pure AEC objective function can exhibit multiple minima with identical AEC values, and hence potentially many solutions that have the same AEC value but different parameter values (β_0, β) are found. Consequently, the solution that is returned by the optimization method depends on the starting values of the parameters in the training step for which we use the coefficients of a standard logistic regression model based on (11). Moreover, Figs. 1a and 2

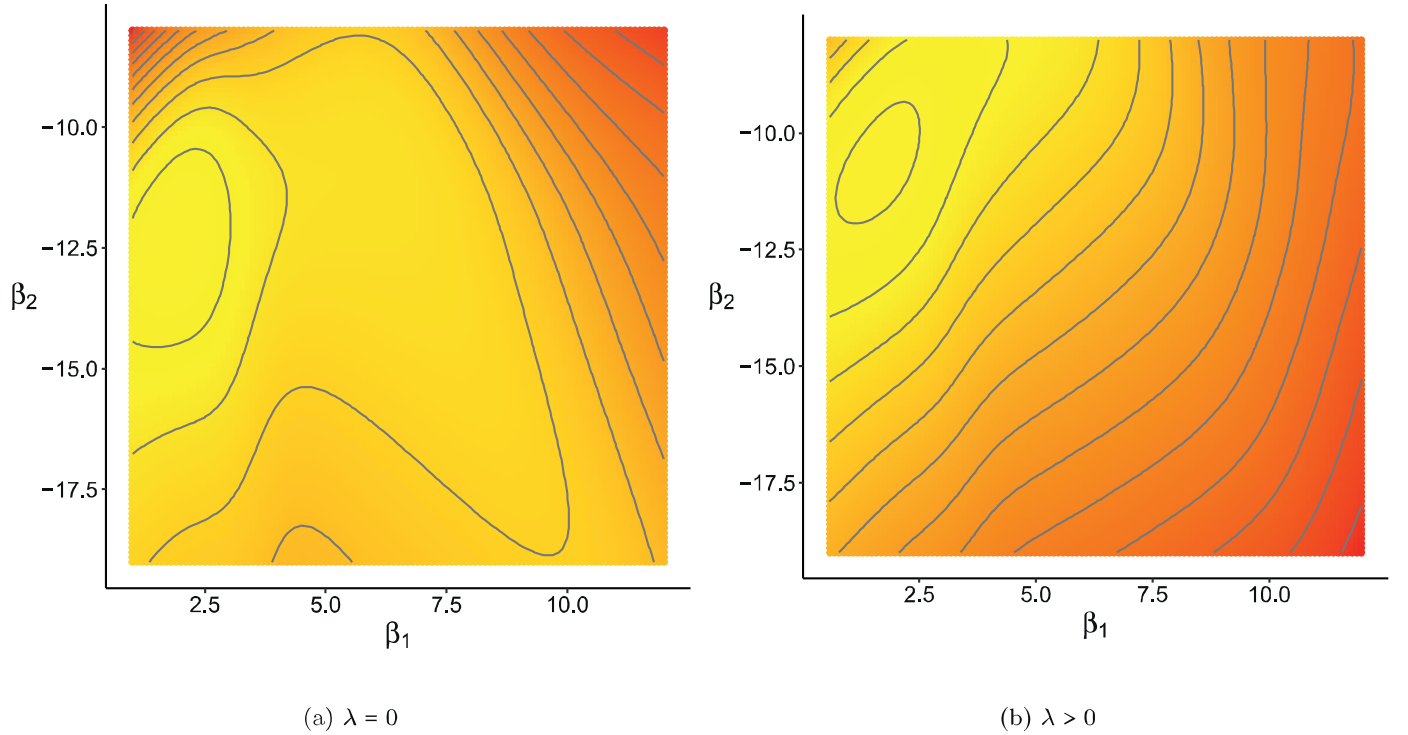


Fig. 1. Landscapes of cslogits objective function based on a simulated data set, where low values are in yellow and high values are in red. If the objective function only consists of the AEC measure (Eq. (14)) with $\lambda = 0$ as in (a), the global minimum is unstable (due to low convexity) and multiple minima with identical AEC values may exist. On the other hand, if objective function (14) is augmented with the lasso penalty ($\lambda > 0$) as in (b), it induces an incline on the surface of the objective function that stabilizes the minimum and thus makes the gradient-based optimization method more efficient. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

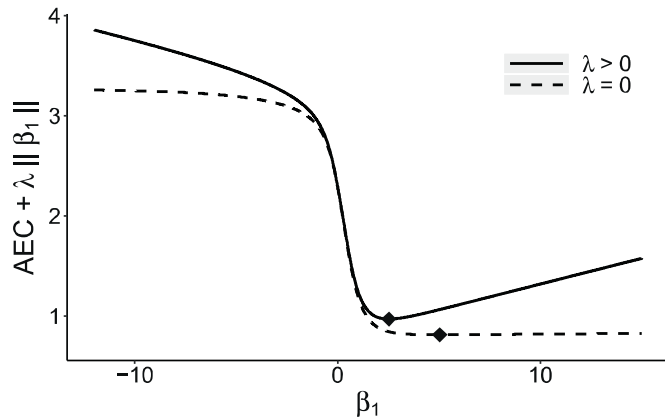


Fig. 2. Objective function of cslogit, with ($\lambda > 0$) and without ($\lambda = 0$) the lasso penalty, using one regression coefficient (β_1). The optimal solution to the AEC objective function ($\lambda = 0$) is very unstable, while the addition of the lasso regularization ($\lambda > 0$) stabilizes the minimum and causes the coefficient to shrink to zero.

(with $\lambda = 0$) illustrate that the optimal solution to the AEC objective function is highly unstable. This means that a large shift in parameter values (β_0, β) still results in AEC values close to the optimum which makes it difficult for the gradient-based optimization method to converge. Therefore, in the spirit of the Empirical Risk Minimization framework of Vapnik (2013), we augment the objective function with a lasso penalty to avoid the undesirable behavior of finding “unstable” solutions. Generally, the lasso regularization penalizes model complexity and biases the gradient-based search toward simpler models as coefficients are shrunk to zero (Tibshirani, 1996) as shown in Fig. 2. Note that we do not claim that the inclusion of the lasso penalty generally results in a unique

minimum (Stripling, vanden Broucke, Antonio, Baesens, & Snoeck, 2018).

Hence we consider the lasso-regularized version of the objective function,

$$Q_{\lambda, Y, X}^c(\beta_0, \beta) = AEC(\beta_0, \beta) + \lambda \|\beta\|_1 \quad (14)$$

where $\lambda \geq 0$ is the regularization parameter and $\|\beta\|_1 = \sum_{j=1}^d |\beta_j|$ is the L_1 -norm of β . Note that the lasso regularization only penalizes the regression coefficients in β - not the intercept β_0 . Clearly, the larger λ , the stronger the lasso penalty. Typically, the predictors are standardized in the lasso model so that they have zero mean (i.e. $\frac{1}{N} \sum_{i=1}^N x_{ij} = 0$) and unit variance (i.e. $\frac{1}{N} \sum_{i=1}^N x_{ij}^2 = 1$) (Hastie, Tibshirani, & Wainwright, 2015). In Figs. 1 and 2, the effect of the lasso regularization on the objective landscape is clearly visible. The inclusion of the penalty term creates an incline on the surface of the objective function that noticeably helps the gradient-based optimization method to find the minimum more efficiently.

The regularization parameter λ cannot be directly estimated from the data and has to be determined by means of hyperparameter optimization strategies. We have opted for a grid search in combination with cross-validation. The optimal value for λ corresponds to the value with the lowest AEC value. Instead of focusing on out-of-sample tuning of λ , one could also associate criteria for in-sample training, such as the Akaike (AIC, Akaike, 1974) or the Bayesian (BIC, Schwarz et al., 1978) information criteria, to the grid search. Another possibility is to experiment with local search methods. Assuming the optimal λ value has been found, the lasso-regularized logistic regression (14) aims to achieve a good balance between minimizing costs and model complexity in which only predictors with sufficiently large predictive power have a nonzero regression coefficient.

The pseudocode for the cslogit algorithm is provided in Algorithm 1. The goal of the algorithm is to find estimates for the re-

Algorithm 1 cslogit.**Inputs**

- $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, data set with d predictors;
- $\mathbf{C} \in \mathbb{R}^{N \times 2}$, cost matrix. For each instance, the first (resp. second) column contains the cost of correctly (resp. wrongly) predicting the binary class of the instance;
- λ , regularization parameter;
- M , maximum number of iterations (default is 10,000);
- Δf , relative tolerance of the objective function (14) (default is $1e-8$);
- $\Delta \beta$, relative tolerance of the regression coefficients (default is $1e-5$);

Starting values for the regression parameters

Fit a regular logistic regression model (11) to the data to obtain initial starting values (β_0^0, β^0) for the optimization.

The main loop of the algorithm uses the gradient based-optimization method from

Kraft (1988, 1994) to sequentially minimize $Q_{\lambda, Y, X}^c(\beta_0, \beta) = AEC(\beta_0, \beta) + \lambda \|\beta\|_1$.

At each iteration $m \leq M$, the current estimate (β_0^m, β^m) is improved by minimizing $Q_{\lambda, Y, X}^c$ using the gradient (13) evaluated at (β_0^m, β^m) .

Termination criteria

The algorithm terminates when $|Q_{\lambda, Y, X}^c(\beta_0^{m-1}, \beta^{m-1}) - Q_{\lambda, Y, X}^c(\beta_0^m, \beta^m)| < \Delta f \cdot Q_{\lambda, Y, X}^c(\beta_0^m, \beta^m)$ or when $|\beta_j^m - \beta_j^{m-1}| < \Delta \beta \cdot \beta_j^m$ for all $j = 0, 1, \dots, d$.

If the algorithm does not converge, it will terminate when the maximum number of iterations is reached in which case a warning message is written to the command line.

Output

Finally, the set of regression parameters $(\hat{\beta}_0, \hat{\beta})$ with the lowest objective value $Q_{\lambda, Y, X}^c$ is returned.

gression coefficients (β_0, β) that minimize the cost-sensitive objective function (14). The algorithm starts by fitting a regular logistic regression model (11) to the provided data set to obtain initial values (β_0^0, β^0) for the optimization. The cslogit algorithm uses the gradient based-optimization method from Kraft (1988, 1994) to sequentially minimize the regularized average expected cost. At each iteration m , the current estimate (β_0^m, β^m) is improved by using the gradient of $Q_{\lambda, Y, X}^c$ (13) evaluated at (β_0^m, β^m) . The algorithm terminates when the objective value converges or all of the regression coefficients converge. The details of the termination criteria are given in Algorithm 1. If convergence does not occur, the algorithm terminates after a user-specified number of iterations, which is set at 10,000 by default. The result of the cslogit algorithm is the set of regression parameters $(\hat{\beta}_0, \hat{\beta})$ with the lowest value for $Q_{\lambda, Y, X}^c$.

The cslogit algorithm is included in the package cslogit. The package is written in R and is available at github.com/SebastiaanHoppner/CostSensitiveLearning. The user interface of cslogit is included in Appendix A of the Supplementary Material.

3.2. Gradient boosted decision trees

Boosting is one of the most powerful learning concepts that has been implemented over the past twenty years. The motivation behind boosting was a process that combines the outputs of many “weak” classifiers to create a strong “committee” (Friedman, Hastie,

& Tibshirani, 2009). A weak classifier, also called a base learner, is one whose performance is only slightly better than random guessing. An example of a weak classifier is a two terminal-node classification tree, also referred to as a tree “stump”. In short, gradient boosting recursively applies the weak classification algorithm to modified versions of the data, so that each additional model is trained to predict the aggregated error of the previously trained models, with the first model trained to predict the original target variable. The predictions of all of them are then combined to produce the final prediction by a weighted majority vote. According to Friedman et al. (2009), trees have one element, namely inaccuracy, which prohibits them from being the perfect instrument for predictive learning. They rarely provide predictive precision similar to the best that can be accomplished with the available data. Boosting decision trees, often dramatically, enhances their precision while retaining most of their desirable data mining characteristics, like the natural handling “mixed” type data, being insensitive to monotone transformations of predictor variables, and having the ability to deal with irrelevant inputs. Some of the benefits of decision trees sacrificed by boosting are speed, interpretability, and potentially robustness against overlapping class distributions and particularly mislabeling of training data. A gradient-boosted model is a tree-boosting generalization that tries to mitigate these issues in order to create an precise and efficient data mining process. Gradient tree boosting is implemented in several R software packages, including gbm (Ridgeway, 1999; 2007) and mboost (Hothorn & Bühlmann, 2006). However, these algorithms use an accuracy related performance measure, like the binomial negative log-likelihood (11), as their objective function. The R package xgboost, on the other hand, is made to be extendible and allows us to easily define our own cost-sensitive objection function.

xgboost is short for eXtreme Gradient Boosting (Chen & Guestrin, 2016). It is an efficient and scalable implementation of the gradient boosting framework by Friedman, Hastie, Tibshirani et al. (2000) and Friedman (2001). Both xgboost and gbm follow the same principle of gradient boosting, but there are some key differences in the modeling details. Specifically, xgboost uses a more regularized model formalization to control over-fitting, which gives it better performance. The name xgboost refers to the engineering goal to push the limit of computational resources for boosted tree algorithms. As a result, it is generally over 10 times faster than gbm (Chen & Guestrin, 2016).

Consider a data set with N instances and d variables $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$ where $|\mathcal{D}| = N$, $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$. A tree ensemble method uses K additive functions to predict the output:

$$\hat{y}_i = \phi(\mathbf{x}_i) = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F},$$

where $\mathcal{F} = \{f(\mathbf{x}) = w_q(\mathbf{x}) \mid q : \mathbb{R}^d \rightarrow T, w \in \mathbb{R}^T\}$ is the space of decision trees. Here q represents the structure of each tree that maps an instance to the corresponding leaf index. T is the number of leaves in the tree. Each f_k corresponds to an independent tree structure q and leaf weights w . For a given instance, the decision rules in the trees (given by q) are used to classify it into the leaves and calculate the final prediction by summing up the weights in the corresponding leaves (given by w). To learn the set of functions used in the model, the following regularized objective is optimized:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \quad (15)$$

where $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$.

Here l is a differentiable convex objective (i.e. loss) function that measures the difference between the prediction $\hat{y}_i$ and the target y_i . The second term Ω penalizes the complexity of the model (i.e., the decision tree functions). The additional regularization term

helps to smooth the final learnt weights to avoid over-fitting. Intuitively, the regularized objective tends to select a model employing simple and predictive functions. When the regularization parameter is set to zero, the objective falls back to the traditional gradient tree boosting. The tree ensemble method (15) includes functions as parameters and cannot be optimized using traditional optimization methods in Euclidean space. Instead, the model is trained in an additive manner. Formally, let $\hat{y}_i^{(t)}$ be the prediction of the i th instance at the t th iteration, then f_t is added to minimize the following objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^N l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t).$$

This means that the f_t that most improves the model according to (15) is greedily added. Second-order approximation can be used to quickly optimize the objective in the general setting (Friedman et al., 2000):

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^N \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t)$$

where

$$g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)}) \text{ and } h_i = \partial_{\hat{y}^{(t-1)}}^2 l(y_i, \hat{y}_i^{(t-1)})$$

are first and second order gradient statistics on the objective function. The constant terms can be removed to obtain the following simplified objective at step t :

$$\tilde{\mathcal{L}}^{(t)} \simeq \sum_{i=1}^N \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i) \right] + \Omega(f_t). \quad (16)$$

Notice that this last equation only contains the gradient and second order gradient of objective function l . In order to achieve an algorithm for instance-dependent cost-sensitive gradient tree boosting, we substitute the gradient and second order gradient of the *average expected cost* (7). Given the provided training data $(Y, \mathbf{X})$ and a set of predictions η_i ($i = 1, \dots, N$) before logistic transformation, the probability scores are defined as

$$s_i = \frac{1}{1 + e^{-\eta_i}} \quad (i = 1, \dots, N).$$

The gradient and second order gradient of the AEC (7) are then ($i = 1, \dots, N$):

$$\begin{aligned} g_i &= \frac{\partial \text{AEC}}{\partial \eta_i} = s_i(1 - s_i) \left[y_i \left(C_i(1|1) - C_i(0|1) \right) \right. \\ &\quad \left. + (1 - y_i) \left(C_i(1|0) - C_i(0|0) \right) \right] \\ &= s_i(1 - s_i) (c_f - y_i A_i), \\ h_i &= \frac{\partial^2 \text{AEC}}{\partial \eta_i^2} = s_i(1 - s_i) (1 - 2s_i) \left[y_i \left(C_i(1|1) - C_i(0|1) \right) \right. \\ &\quad \left. + (1 - y_i) \left(C_i(1|0) - C_i(0|0) \right) \right] \\ &= s_i(1 - s_i) (1 - 2s_i) (c_f - y_i A_i) \\ &= \frac{\partial \text{AEC}}{\partial \eta_i} (1 - 2s_i). \end{aligned} \quad (17)$$

The csboost algorithm is essentially a wrapper for the `xgb.train` function from the `xgboost` package in R. The details are discussed below. The `xgboost` implementation allows us to specify the average expected cost (7) as both the objective function and evaluation metric. In the case of csboost, the training dataset is specified using a formula and a data frame as is most common in R implementations, rather than using an `xgb.DMatrix` as is the case with `xgb.train`. This makes the csboost function more user-friendly.

The csboost algorithm is included in the package `csboost`. The package is written in R and is available at github.com/SebastiaanHoppner/CostSensitiveLearning. The user interface of csboost is included in Appendix B of the Supplementary Material.

4. Experiments

We assess the performance of our new methods by benchmarking them against their cost-insentitive counterparts, (regular) logistic regression and the `xgboost` algorithm. To do so, we apply the classification techniques to a publicly available real-life fraud data set and a data set provided by a large bank. All methods are evaluated using Savings, Expected Savings, Precision, Recall and F_1 measure where we use the instance-dependent thresholds (2).

4.1. Data sets

The first data set is the Credit Card Transaction Data available at kaggle.com/mlg-ulb/creditcardfraud. The data consists of transactions made by credit cards in September 2013 by European cardholders. This data set presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The data set is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions. It contains only numerical input variables which are the result of a PCA transformation. Due to confidentiality issues, the original features and more background information about the data cannot be provided. Features V1, V2, ..., V28 are the principal components obtained with PCA. The only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the data set. The feature 'Amount' is the transaction amount where we remove a few transactions with a zero amount. Feature 'Class' is the response variable which takes value 1 in case of fraud and 0 otherwise. We select the features V1, V2, ..., V28 and the logarithmically transformed Amount as predictor variables for the classification methods.

The second data set has been provided to our research group by a large bank and will be referred to as the Bank data set. The data contains 31,763 transactions made between September 2018 and July 2019. In total there are 506 (1.6%) transactions labeled as fraudulent. The data set contains 21 numerical input features, 3 categorical features and the fraud indicator as response variable.

4.2. Experimental design

For each data set, we perform 5 replications of two-fold cross validation (5×2 cv) (Demšar, 2006; Dietterich, 1998; Höppner et al., 2018). In each replication, we randomly partition the data into two, non-overlapping sets (also called folds) S_1 and S_2 , so that both sets are equal size. We then train each classifier on S_1 and evaluate on S_2 , followed by training on S_2 and evaluating on S_1 . This procedure is repeated five times resulting in a total of ten (5×2) out-of-sample classification performance estimates.

For the experiments, we want to ensure that in each of the five iterations both folds contain an equal balance of high, middle and low value fraud cases. Therefore, we compute the 33% and 66% quantiles of the fraudulent amounts, and we divide every transfer in one of three categories (i.e. high, middle or low) depending on their amount with respect to these quantiles. For the Credit Card Transaction Data, transfers with an amount below € 1.10 are categorized as "low", transfers between € 1.10 and € 99.99 are considered "middle", and transfers above € 99.99 are categorized as "high". Similarly for the Bank data set, the 33% and 66% quantiles of the fraudulent amounts are € 1999.33 and € 5000, respectively. Using these three categories, each fold in the cross validation procedure is stratified according to the binary response variable as well as the amount category in order to obtain similar distributions in the folds as observed in the original data set.

After the training and testing set are defined, we scale the (non-categorical) predictors in the training fold to zero mean and unit

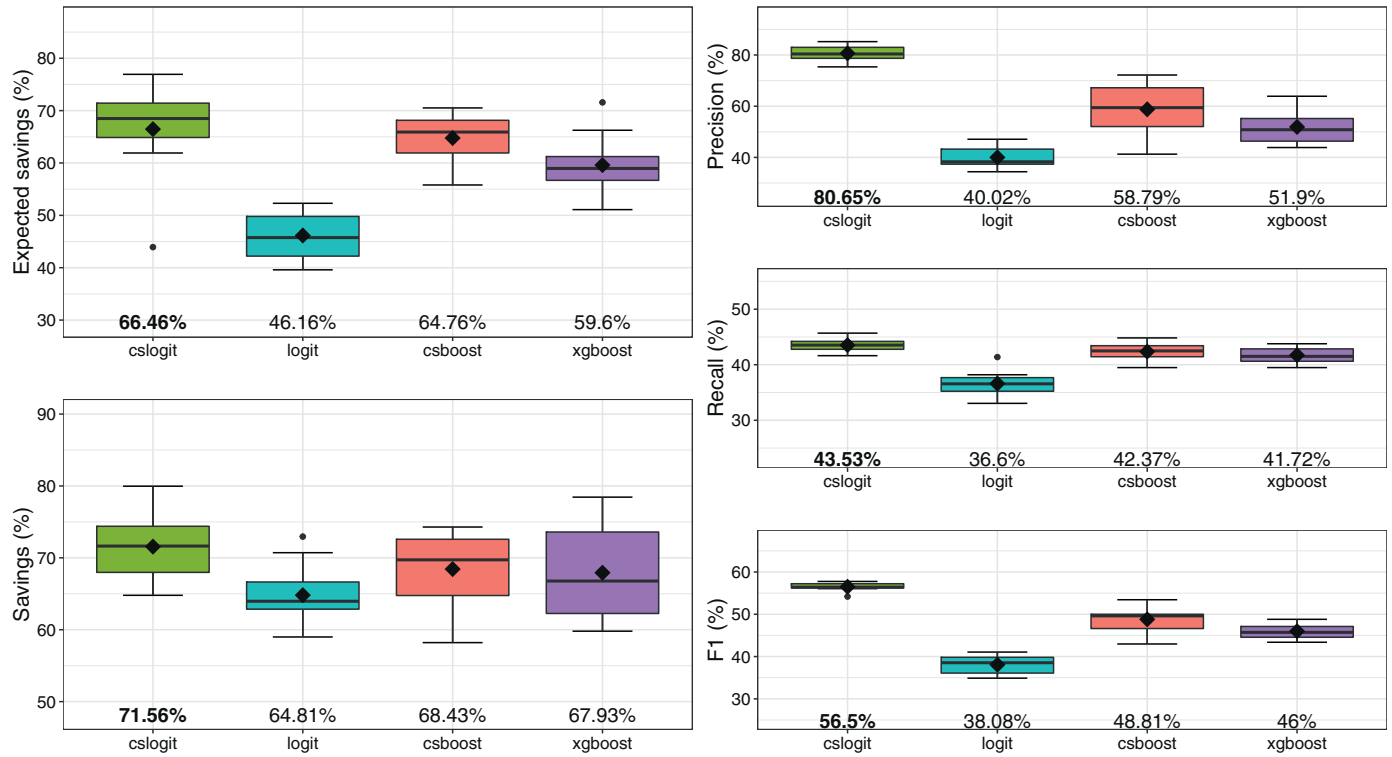


Fig. 3. Five times two-fold cross validation results for the credit card transaction data. Each boxplot is based on 10 (5×2) out-of-sample performance estimates over the 10 test sets. In addition, the mean ($\blacklozenge$) of these 10 estimates is stated at the bottom in percent.

Table 2

Average time (in seconds) to fit each of the classification methods on the training set of the Credit Card Transaction Data (top) and the Bank data (bottom), and the size of the respective training sets.

	Size of training fold		Average computation time			
	# instances	# features	cslogit	logit	csboost	xgboost
Credit card data	141,491	29	4.82	2.94	5.65	9.48
Bank data	15,881	24	0.64	0.44	0.89	1.15

variance. The corresponding predictors in the testing fold are then scaled using the same mean and variance estimates from the training fold. To keep the analysis of the data sets manageable, we only consider main effects in the models and we do not include interactions of any degree.

4.3. Results

Fig. 3 contains the results of the 5×2 -fold cross validation procedure for the Credit Card Transaction Data. We measure each classifier's performance over the ten (5×2) test sets, so each boxplot is based on 10 out-of-sample estimates of performance: Savings, Expected Savings, Precision, Recall and F_1 measure where we use the instance-dependent thresholds (2). We compare the classifiers based on their average performance which is stated below each boxplot in percent. In terms of expected savings, cslogit outperforms logistic regression and csboost outperforms gradient boosted trees (xgboost) as can be seen in the top left figure. Of course, this is expected since both cost-sensitive methods have the average expected cost (AEC) in their objective function which they minimize. Note that by minimizing the AEC (7), the expected savings measure is maximized (8). After applying the instance-dependent cost-related threshold (2), the savings measure can be assessed. Both cost-sensitive methods outperform their classical counterparts although the difference between them is smaller. Although cslogit and csboost are designed to optimize cost-related measures, like

AEC and expected savings, it is interesting to notice that cslogit and csboost do better than logistic regression and xgboost in terms of precision, recall and F_1 on this data set. Overall, the difference in performance between cslogit and logit is larger than the difference between csboost and xgboost. This is also reflected in their cost: on average across the 10 folds, the difference in total costs between cslogit and logit is € 2005.83 while the difference in total costs between csboost and xgboost is € 169.44. The average execution times are reported in Table 2. These execution times were measured on an Intel core i5 with 2.7 GHz and 8 GB RAM.

Fig. 4 contains the results of the 5×2 -fold cross validation procedure for the Bank data set. As expected, cslogit and csboost largely outperform logit and xgboost in terms of expected savings. When applying the instance-dependent cost-related threshold (2), the predicted fraud probabilities are converted into binary decisions (i.e. fraud or not), on which the savings measure is computed. Compared to the previous data set, cslogit slightly outperforms logistic regression while the difference between csboost and xgboost is larger on average. The average difference between cslogit and logit in terms of average cost for a single transfer between is € 0.62. Although this difference might seem small, it can accumulate to large amounts if we take into account that a bank may process over 100,000 transactions each day. The average difference between csboost and xgboost in terms of average cost for a single transaction is € 3.25. cslogit and csboost are designed to minimize the financial losses due to fraud by taking into account

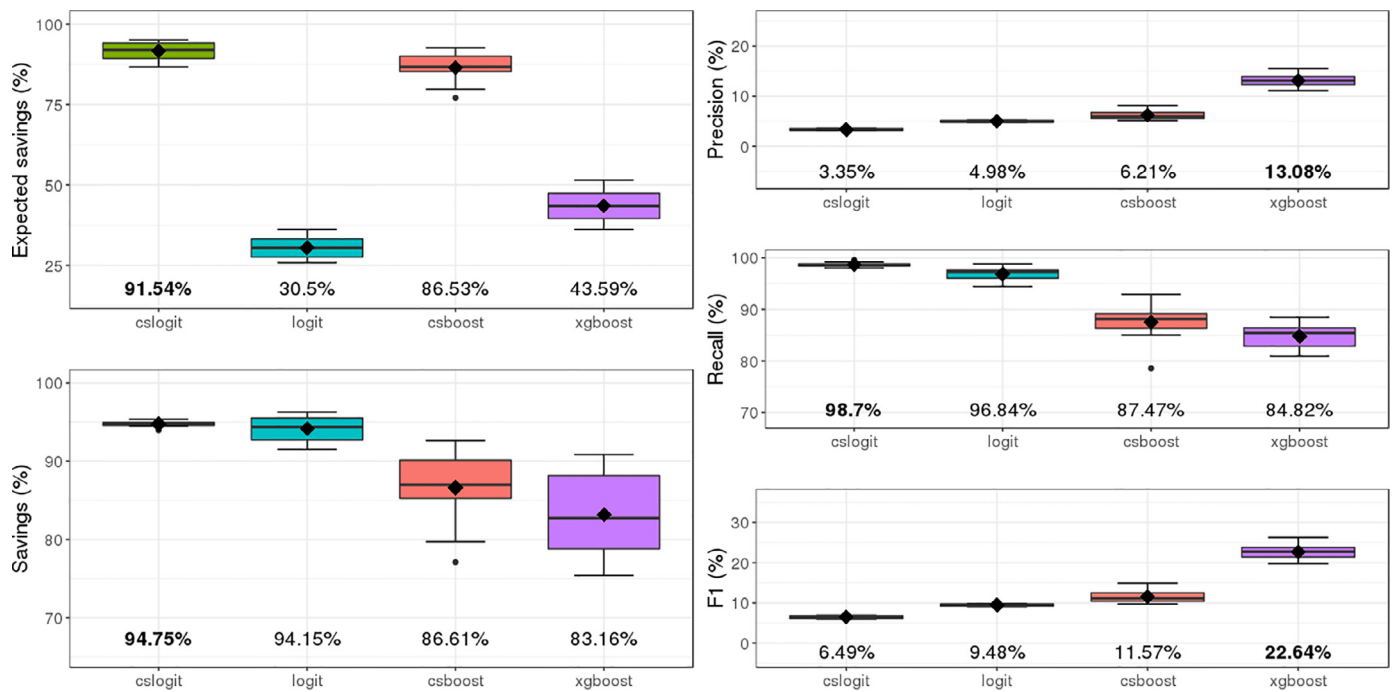


Fig. 4. Five times two-fold cross validation results for the bank transaction data. Each boxplot is based on 10 (5×2) out-of-sample performance estimates over the 10 test sets. In addition, the mean (♦) of these 10 estimates is stated at the bottom in percent.

the various costs between transactions due to classification. The results in Fig. 4 illustrate that model selection purely based on accuracy related performance measures, such as precision, recall and F_1 , likely results in models that have a higher cost. This is clearly demonstrated by the fact that cslogit has the highest cost savings while simultaneously has the worst F_1 values.

5. Conclusions and future research

In this paper, we presented two new classifiers called cslogit and csboost which are based on lasso-regularized logistic regression and gradient tree boosting, respectively. Each method directly minimizes the proposed instance-dependent cost measure in the model construction step. As a result, cslogit and csboost aim to create the detection model which minimizes the financial loss due to fraud. Furthermore, based on the instance-dependent cost matrix for transfer fraud, we derived a transfer-specific threshold that allows for making the optimal cost-based decision for each transaction.

In our benchmark study, cslogit and csboost outperform their classical counterpart models, which are ignorant of any classification costs, in terms of the costs saved due to detecting fraud. We conclude that model selection based on accuracy related measures, such as precision and recall, leads to more costly results. In this paper, we have shown that our proposed methods align best with the core business objection of cost minimization by prioritizing the detection of high-amount fraudulent transfers.

Concerning future research, we intend to include artificial neural networks to the collection of instance-dependent cost-sensitive classifiers. For this, we can turn to the work done by Vapnik (2013) which combines concepts from the Empirical Risk Minimization framework with the domain of neural networks. The resulting method, called csnet, computes the gradient of the proposed instance-dependent cost measure in its backpropagation algorithm. An extensive empirical evaluation and comparison between the cost-sensitive methods cslogit, csboost and csnet can then be conducted. Although cslogit and csboost are used for de-

tecting card transaction fraud, the framework that is presented in this paper, including both methods, can deal with any cost matrix. Therefore, cslogit and csboost have potential in multiple fraud detection domains such as insurance fraud (Dionne, Giuliano, & Picard, 2009), e-commerce fraud (Nanduri, Jia, Oka, Beaver, & Liu, 2020) and social security fraud (Van Vlasselaer, Eliassi-Rad, Akoglu, Snoeck, & Baesens, 2017), as well as other analytical tasks where costs are important such as credit-risk evaluation (Baesens, Setiono, Mues, & Vanthienen, 2003; Verbraken, Bravo, Weber, & Baesens, 2014) and customer churn prediction (Verbeke et al., 2012). The main adaptation to these tasks would consist of identifying the costs of classifying instances and defining the appropriate cost matrix. An example is included in Appendix C where we adapt cslogit and csboost for the task of predicting credit-risk.

Acknowledgements

This work was supported by the BNP Paribas Fortis Chair in Fraud Analytics and Internal Funds KU Leuven under Grant C16/15/068.

Supplementary material

Supplementary material associated with this article can be found, in the online version, at [10.1016/j.ejor.2021.05.028](https://doi.org/10.1016/j.ejor.2021.05.028)

References

- Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716–723.
- Baesens, B., Setiono, R., Mues, C., & Vanthienen, J. (2003). Using neural network rule extraction and decision tables for credit-risk evaluation. *Management Science*, 49(3), 312–329.
- Bahnsen, A. C., Aouada, D., & Ottersten, B. (2014a). Example-dependent cost-sensitive logistic regression for credit scoring. In *2014 13th international conference on machine learning and applications* (pp. 263–269). IEEE.
- Bahnsen, A. C., Aouada, D., Stojanovic, A., & Ottersten, B. (2016). Feature engineering strategies for credit card fraud detection. *Expert Systems with Applications*, 51, 134–142.

- Bahnens, A. C., Stojanovic, A., Aouada, D., & Ottersten, B. (2014b). Improving credit card fraud detection with calibrated probabilities. In *Proceedings of the 2014 siam international conference on data mining* (pp. 677–685). SIAM.
- Bahnens, A. C., Villegas, S., Aouada, D., Ottersten, B., Correa, A., & Villegas, S. (2017). Fraud detection by stacking cost-sensitive decision trees. *Data Science for Cyber-Security*.
- Chan, P. K., & Stolfo, S. J. (1998). Toward scalable learning with non-uniform class and cost distributions: A case study in credit card fraud detection. In *KDD* (pp. 164–168).
- Chen, T., & Guestrin, C. (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794). ACM.
- Dal Pozzolo, A., Caelen, O., Le Borgne, Y.-A., Waterschoot, S., & Bontempi, G. (2014). Learned lessons in credit card fraud detection from a practitioner perspective. *Expert Systems With Applications*, 41(10), 4915–4928.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine learning research*, 7(Jan), 1–30.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923.
- Dionne, G., Giuliano, F., & Picard, P. (2009). Optimal auditing with scoring: Theory and application to insurance fraud. *Management Science*, 55(1), 58–70.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *International joint conference on artificial intelligence*: 17 (pp. 973–978). Lawrence Erlbaum Associates Ltd.
- European Central Bank, E. (2018). Fifth report on card fraud. URL www.ecb.europa.eu/pub/cardfraud/html/ecb.cardfraudreport201809.en.html.
- Friedman, J., Hastie, T., & Tibshirani, R. (2009). *The elements of statistical learning: Data mining, inference, and prediction*: 2. New York: Springer series in statistics.
- Friedman, J., Hastie, T., Tibshirani, R., et al. (2000). Additive logistic regression: A statistical view of boosting (with discussion and a rejoinder by the authors). *The Annals of Statistics*, 28(2), 337–407.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 1189–1232.
- Hand, D. J., Whitrow, C., Adams, N. M., Juszczak, P., & Weston, D. (2008). Performance criteria for plastic card fraud detection tools. *Journal of the Operational Research Society*, 59(7), 956–962.
- Hastie, T., Tibshirani, R., & Wainwright, M. (2015). *Statistical learning with sparsity: The lasso and generalizations*. CRC press.
- Höppner, S., Stripling, E., Baesens, B., vanden Broucke, S., & Verdonck, T. (2018). Profit driven decision trees for churn prediction. *European Journal of Operational Research*.
- Hosmer Jr, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*: 398. John Wiley & Sons.
- Hothorn, T., & Bühlmann, P. (2006). Model-based boosting in high dimensions. *Bioinformatics*, 22(22), 2828–2829.
- Kraft, D. (1988). A software package for sequential quadratic programming. *Forschungsbericht- Deutsche Forschungs- und Versuchsanstalt für Luft- und Raumfahrt*.
- Kraft, D. (1994). Algorithm 733: Tomp–fortran modules for optimal control calculations. *ACM Transactions on Mathematical Software (TOMS)*, 20(3), 262–281.
- Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT press.
- Nanduri, J., Jia, Y., Oka, A., Beaver, J., & Liu, Y.-W. (2020). Microsoft uses machine learning and optimization to reduce e-commerce fraud. *Interfaces*, 50(1), 64–79.
- Ngai, E. W., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011). The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature. *Decision Support Systems*, 50(3), 559–569.
- Phua, C., Lee, V., Smith, K., & Gayler, R. (2010). A comprehensive survey of data mining-based fraud detection research. arXiv:1009.6119.
- Ridgeway, G. (1999). The state of boosting. *Computing Science and Statistics*, 172–181.
- Ridgeway, G. (2007). Generalized boosted models: A guide to the GBM package. *Update*, 1(1), 2007.
- Sahin, Y., Bulkan, S., & Duman, E. (2013). A cost-sensitive decision tree approach for fraud detection. *Expert Systems with Applications*, 40(15), 5916–5923.
- Schwarz, G., et al. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461–464.
- Stripling, E., vanden Broucke, S., Antonio, K., Baesens, B., & Snoeck, M. (2018). Profit maximizing logistic model for customer churn prediction using genetic algorithms. *Swarm and Evolutionary Computation*, 40, 116–130.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288.
- Van Vlasselaer, V., Eliassi-Rad, T., Akoglu, L., Snoeck, M., & Baesens, B. (2017). Gotcha! network-based fraud detection for social security fraud. *Management Science*, 63(9), 3090–3110.
- Vapnik, V. (2013). *The nature of statistical learning theory*. Springer science & business media.
- Verbeke, W., Dejaeger, K., Martens, D., Hur, J., & Baesens, B. (2012). New insights into churn prediction in the telecommunication sector: A profit driven data mining approach. *European Journal of Operational Research*, 218(1), 211–229.
- Verbraken, T., Bravo, C., Weber, R., & Baesens, B. (2014). Development and application of consumer credit scoring models using profit-based classification measures. *European Journal of Operational Research*, 238(2), 505–513.
- Whitrow, C., Hand, D. J., Juszczak, P., Weston, D., & Adams, N. M. (2009). Transaction aggregation as a strategy for credit card fraud detection. *Data Mining and Knowledge Discovery*, 18(1), 30–55.
- Zadrozny, B., & Elkan, C. (2001). Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers. In *ICML: 1* (pp. 609–616). Citeseer.